

Tarea 07: Juego del Gato - Tic-Tac-Toe

Instrucciones

Desarrolla una aplicación móvil implementando el Framework [Flutter](#) que funcione como el juego del gato (Tic-Tac-Toe) en modalidad multijugador en tiempo real. Demostrando comunicaciones entre dispositivos, persistencia de datos en un servicio externo y una experiencia de usuario (UX) pulida y funcional.

Objetivo

Evaluar la capacidad del equipo para integrar servicios externos de autenticación y base de datos en tiempo real dentro de una aplicación móvil, así como su habilidad para gestionar la sincronización de estado entre dos dispositivos distintos de forma coherente y sin conflictos.

Estilo visual

El estilo visual y diseño de la aplicación queda a decisión del equipo; se busca un estilo visual con personalidad, calidad, calidez y visualmente atractiva.

Arquitectura y servicios externos

La aplicación debe conectarse obligatoriamente a un servicio backend externo para manejar tanto la autenticación de usuarios como la persistencia y sincronización de partidas. Se recomienda el uso de Firebase (Authentication + Firestore), aunque solo es una recomendación, el equipo puede utilizar cualquier otro servicio.

Vistas

Vista: Registro e inicio de sesión

Es la primera vista que verá un usuario nuevo. Debe ser clara, sencilla y transmitir confianza.

- **Identidad institucional:** Debe incluir el logo de la Universidad de Sonora.

- **Título:** debe mostrar un nombre creativo relacionado con el juego.
- **Crédito:** Nombre completo de los miembros del equipo.
- **Registro:** Formulario con los campos; nombre de usuario, correo electrónico y contraseña. Debe incluir validación básica de formato (correo, válido, contraseña de al menos 6 caracteres).
- **Inicio de sesión:** Formulario con correo y contraseña para usuarios ya registrados.
- **Navegación:** Botones claros para alternar entre el formulario de registro y el de inicio de sesión.

Vista: Sala de espera

Esta es la vista que se verá después del inicio de sesión. Desde aquí el usuario gestiona sus partidas.

- **Saludo:** Debe mostrarse el nombre del usuario autenticado.
- **Crear partida:** Botón para crear una nueva sala de juego. Al crearse, se debe generar y mostrar un código de sala único (ej. un código de 6 caracteres) que el usuario puede compartir con su oponente.
- **Unirse:** Campo de texto donde el usuario puede ingresar el código de sala de otra persona para unirse como segundo jugador.
- **Cerrar sesión:** Botón para cerrar la sesión del usuario actual.

Vista: Juego

En esta vista ocurre la interacción principal.

- **Tablero:** Cuadrícula 3x3 que represente el estado actual del juego. Cada celda debe ser interactiva únicamente durante el turno del jugador correspondiente.
- **Indicadores:** El usuario debe poder ver en todo momento:
 - Símbolo asignado a cada jugador (X u O).
 - De quién es el turno actual ("Tu turno" o "Turno del oponente").
 - Nombre de usuario del oponente.
- **Sincronización en tiempo real:** Cada movimiento realizado por un jugador debe reflejarse inmediatamente en el dispositivo del oponente sin necesidad de recargar ni intervenir manualmente.

- **Detección de fin de partida:** La aplicación debe detectar automáticamente las tres condiciones de fin; victoria, derrota y empate. Al ocurrir cualquiera de ellas debe mostrarse un mensaje o animación clara que se lo indique a los jugadores.
- **Animaciones:** Cada marca o símbolo asignado al jugador en el tablero debe aparecer con una animación. La línea ganadora debe destacarse visualmente al terminar la partida.

Vista: Resultados

En esta vista se muestran los resultados de la partida.

- **Resultado:** Debe indicar claramente si el jugador ganó, perdió o empató.
- **Resumen de la partida:** Debe mostrar los nombres de ambos jugadores y el resultado final.
- **Acciones:** Dos botones: “Volver a jugar” (regresa a la sala de espera) y “Cerrar sesión” (cierra la sesión del usuario actual).

Vista: Dashboard (Puntos extra)

En esta vista opcional, se debe mostrar una tabla con el Top 10 mejores jugadores.

- **Tabla de clasificación:** Lista de los 10 jugadores con más victorias registradas en la base de datos, mostrando para cada uno: posición, nombre de usuario, número de victorias y número de partidas jugadas.
- **Actualización:** Los datos deben obtenerse desde la base de datos y reflejar el estado actual del historial de partidas guardadas.
- **Acceso:** Debe existir un botón o ícono en la sala de espera que permita navegar hasta esta vista.

Persistencia de datos

Cada partida completada debe quedar registrada en la base de datos externa con la siguiente información:

- Nombres de los jugadores.

- Lista ordenada de movimientos realizados durante la partida, donde cada movimiento registre: el jugador que realizó el movimiento y la celda seleccionada (fila, columna).
- Resultado de la partida: ¿Quién fue el ganador?

¿Qué se califica?

- **Arquitectura del Código:** Se evaluará la separación entre el interfaz (UI) y la lógica del temporizador y los ciclos.
- **Integración con el servicio externo:** Registro, autenticación, sincronización en tiempo real y persistencia de partidas funcionando correctamente.
- **Animaciones de estado:** Las marcas del tablero, la línea ganadora y las transiciones entre pantallas deben estar animadas.
- **Diseño y Coherencia:** La aplicación debe tener una paleta de colores definida (usa [ColorHunt](#)), con tonos distintos para el modo trabajo y el modo descanso que faciliten la lectura y comuniquen el estado de forma intuitiva.
- **Personalización:** La aplicación debe tener un icono personalizado (no el de Flutter) y un nombre de paquete adecuado.
- **Repositorio:** El repositorio del código debe tener un formato adecuado, ser legible, visualmente atractivo y coherente.

Entregables

Para acreditar la tarea se deben entregar lo siguiente:

GitHub

Enlace a un repositorio GitHub que debe contener todo el código fuente. El repositorio debe incluir:

- Descripción de la aplicación
- Tecnologías utilizadas (descripciones y versiones)
- Imágenes de la aplicación

- Las imágenes de la aplicación, pantalla, iconos deben estar ordenadas y escaladas adecuadamente para ser legibles. La calidad del [README.md](#) será tomada en cuenta para la calificación final.
- Release 1.0 (APK para instalar la aplicación)

Es importante aclarar que también se calificará la organización de Readme, su diseño y contenido.

Presentación

Se debe realizar una presentación en vivo de la aplicación funcionando en un celular (No se aceptan máquinas virtuales o simuladores), dicha presentación debe estar acompañada de una presentación en PowerPoint, Canva o cualquier herramienta de su preferencia (**siempre que no esté hecho con IA**).

Se calificará lo siguiente:

- Diseño de la aplicación (ícono, vistas, animaciones, etc.).
- Diseño de la presentación.
- Funcionalidad.
- Pruebas de escritorio.
- Conocimiento de la aplicación desarrollada.

Al mejor diseño

Después de las presentaciones se votará por la mejor aplicación, es decir, la más intuitiva, bonita y funcional. El ganador de la votación recibirá un **punto especial**, al final del semestre, el equipo con más puntos especiales quedará exento de hacer el proyecto final.

Consideraciones

Si durante la presentación el programa no funciona como debería, no cumple con los requisitos mínimos establecidos en las instrucciones u ocurre algún problema relacionado con el código, la calificación final será 0.

Considere hacer pruebas unitarias, pruebas de escritorio, tener todo listo y funcional al menos dos días antes de la presentación para evitar problemas. No se permitirá corregir código o arreglar pequeños errores durante la presentación.